

✓ Model Building

✓ Libraries and Modules

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import transforms, models, datasets
```

✓ Preprocessing

```
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(
        mean = [0.485, 0.456, 0.406],
        std = [0.229, 0.224, 0.225]
    )
])

test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean = [0.485, 0.456, 0.406],
        std = [0.229, 0.224, 0.225]
    )
])
```

✓ Datasets

```
train_dataset = datasets.CIFAR10(
    root = 'data',
    train = True,
    download = True,
    transform = train_transform
)

test_dataset = datasets.CIFAR10(
    root = 'data',
    train = False,
    download = True,
    transform = test_transform
)
```

```

)

train_loader = DataLoader(
    dataset = train_dataset,
    batch_size = 64,
    shuffle = True,
    num_workers = 2
)

test_loader = DataLoader(
    dataset = test_dataset,
    batch_size = 64,
    shuffle = False,
    num_workers = 2
)

```

100%|██████████| 170M/170M [00:13<00:00, 12.7MB/s]

✓ Model Backbone

```

model = models.resnet18(pretrained = True)
for param in model.parameters():
    param.requires_grad = False ## To prevent model weights from being updated

```

```

/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: User
warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: User
warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to /r
100%|██████████| 44.7M/44.7M [00:00<00:00, 214MB/s]

```

✓ Model Training

✓ Updating FC layer

```

device = 'cuda' if torch.cuda.is_available() else 'cpu'

num_classes = 10

model.fc = nn.Linear(model.fc.in_features, num_classes)
model = model.to(device)

ct = nn.CrossEntropyLoss()
op = torch.optim.Adam(model.fc.parameters(), lr = 0.001)

epochs = 5
for epoch in range(epochs):

```

```

model.train()
running_loss = 0.0
for x_batch, y_batch in train_loader:
    x_batch = x_batch.to(device)
    y_batch = y_batch.to(device)

    op.zero_grad()
    preds = model(x_batch)
    loss = ct(preds, y_batch)

    if torch.isnan(loss):
        print("Loss is NaN")
        break

    loss.backward()
    op.step()
    running_loss += loss.item()
print(f"Epoch [{epoch+1}/{epochs}] Loss: {running_loss/len(train_loader):.4}

```

```

Epoch [1/5] Loss: 0.8263
Epoch [2/5] Loss: 0.6180
Epoch [3/5] Loss: 0.5898
Epoch [4/5] Loss: 0.5814
Epoch [5/5] Loss: 0.5655

```

✓ Updating End layer

```

for param in model.layer4.parameters():
    param.requires_grad = True

op = torch.optim.Adam(model.parameters(), lr = 1e-4)

for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    for x_batch, y_batch in train_loader:
        x_batch = x_batch.to(device)
        y_batch = y_batch.to(device)
        preds = model(x_batch)
        loss = ct(preds, y_batch)
        loss.backward()
        op.step()
        op.zero_grad()

        running_loss += loss.item()

    print(f"Epoch [{epoch+1}/{epochs}] Loss: {running_loss/len(train_loader):.4}

```

```

Epoch [1/5] Loss: 0.3845
Epoch [2/5] Loss: 0.1879
Epoch [3/5] Loss: 0.1197
Epoch [4/5] Loss: 0.0795
Epoch [5/5] Loss: 0.0640

```

```
torch.save(model.state_dict(), "weights.pth")
```

✓ Model Evaluation

✓ Eval #1

```
model.eval()
correct = 0
total = 0

with torch.no_grad():
    for x_batch, y_batch in test_loader:
        x_batch = x_batch.to(device)
        y_batch = y_batch.to(device)
        preds = model(x_batch)
        _, predicted = torch.max(preds, 1)
        total += y_batch.size(0)
        correct += (predicted == y_batch).sum().item()

    print(f"Test Accuracy: {100 * correct / total:.2f}%")
```

Test Accuracy: 91.50%

Corruption Functions

```
import torchvision.transforms.functional as F
import random
from PIL import Image, ImageFilter
import io

def gaussian_noise(img, std=0.1):
    tensor = F.to_tensor(img)
    noise = torch.randn_like(tensor) * std
    noisy = torch.clamp(tensor + noise, 0, 1)
    return F.to_pil_image(noisy)

def blur(img, radius=2):
    return img.filter(ImageFilter.GaussianBlur(radius))

def change_brightness(img, factor=0.5):
    return F.adjust_brightness(img, factor)

def jpeg_compress(img, quality=30):
    buffer = io.BytesIO()
    img.save(buffer, format="JPEG", quality=quality)
    buffer.seek(0)
    return Image.open(buffer)
```

Corrupted Dataset Class

```
class CorruptedCIFAR10(datasets.CIFAR10):
    def __init__(self, corruption_fn, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.corruption_fn = corruption_fn

    def __getitem__(self, index):
        img, label = super().__getitem__(index)
        img = self.corruption_fn(img)
        return test_transform(img), label
```

Corrupted datasets

```
noise_dataset = CorruptedCIFAR10(
    corruption_fn=lambda img: gaussian_noise(img, std=0.2),
    root="data",
    train=False,
    download=True
)
noise_loader = DataLoader(noise_dataset, batch_size=64, shuffle=False)

blur_dataset = CorruptedCIFAR10(
    corruption_fn=lambda img: blur(img, radius=2),
    root="data",
    train=False,
    download=True
)
blur_loader = DataLoader(blur_dataset, batch_size=64, shuffle=False)

brightness_dataset = CorruptedCIFAR10(
    corruption_fn=lambda img: change_brightness(img, factor=0.5),
    root="data",
    train=False,
    download=True
)
brightness_loader = DataLoader(brightness_dataset, batch_size=64, shuffle=False)

compression_dataset = CorruptedCIFAR10(
    corruption_fn=lambda img: jpeg_compress(img, quality=30),
    root="data",
    train=False,
    download = True
)
compression_loader = DataLoader(compression_dataset, batch_size=64, shuffle=False)
```

✓ Metrics

```
def compute_metrics(logits):
    probs = torch.softmax(logits, dim =1)
    confidence,_ = torch.max(probs, dim = 1)
    entropy = -torch.sum(probs * torch.log(probs + 1e-8), dim = 1) ## for avoid
    return confidence, entropy
```

✓ Evaluation under corruption

```
def eval2(model, loader, entropy_threshold = 1.5):
    model.eval()

    total = 0
    correct = 0
    rejected = 0
    entropy_list = []
    confidence_list = []

    with torch.no_grad():
        for x, y in loader:
            x = x.to(device)
            y = y.to(device)

            logits = model(x)
            conf, ent = compute_metrics(logits)
            preds = logits.argmax(dim = 1)

            total += y.size(0)
            correct += (preds == y).sum().item()

            entropy_list.append(ent.mean().item())
            confidence_list.append(conf.mean().item())
            rejected += (ent > entropy_threshold).sum().item()

    return {
        'accuracy': 100 * correct / total,
        'avg_entropy': sum(entropy_list) / len(entropy_list),
        'avg_confidence': sum(confidence_list) / len(confidence_list),
        'rejection_rate': rejected / total
    }
```

✓ Gaussian Noise

```
gn = eval2(model, noise_loader, entropy_threshold=1.5)
print(gn)
```

```
{'accuracy': 12.19, 'avg_entropy': 0.7070902745435192, 'avg_confidence': 0.712
```

✓ Image blur

```
ib = eval2(model, blur_loader, entropy_threshold=1.5)
print(ib)
```

```
{'accuracy': 19.83, 'avg_entropy': 0.8805409810345644, 'avg_confidence': 0.685
```

✓ Brightness Variation

```
bv = eval2(model, brightness_loader, entropy_threshold=1.5)
print(bv)
```

```
{'accuracy': 85.26, 'avg_entropy': 0.22725884611629377, 'avg_confidence': 0.91
```

✓ JPEG compression

```
jc = eval2(model, compression_loader, entropy_threshold=1.5)
print(jc)
```

```
{'accuracy': 57.55, 'avg_entropy': 0.48101341135942255, 'avg_confidence': 0.82
```

Although accuracy is low, avg entropy is low and the confidence is high ->

```
def softmax_with_temperature(logits, T=2.0):
    return torch.softmax(logits / T, dim=1)

def compute_metrics2(logits, T=2.0):
    probs = softmax_with_temperature(logits, T)
    confidence, _ = torch.max(probs, dim=1)
    entropy = -torch.sum(probs * torch.log(probs + 1e-8), dim=1)
    return confidence, entropy

def eval3(model, loader, entropy_threshold = 1.5):
    model.eval()

    total = 0
    correct = 0
    rejected = 0
    entropy_list = []
    confidence_list = []

    with torch.no_grad():
        for x, y in loader:
            x = x.to(device)
            y = y.to(device)

            logits = model(x)
            conf, ent = compute_metrics2(logits)
            preds = logits.argmax(dim = 1)
```

```

        total += y.size(0)
        correct += (preds == y).sum().item()

        entropy_list.append(ent.mean().item())
        confidence_list.append(conf.mean().item())
        rejected += (ent > entropy_threshold).sum().item()

    return {
        'accuracy': 100 * correct / total,
        'avg_entropy': sum(entropy_list) / len(entropy_list),
        'avg_confidence': sum(confidence_list) / len(confidence_list),
        'rejection_rate': rejected / total
    }

# Updated evaluations

gn = eval3(model, noise_loader, entropy_threshold=1.5)
print(gn)

ib = eval2(model, blur_loader, entropy_threshold=1.5)
print(ib)

bv = eval2(model, brightness_loader, entropy_threshold=1.5)
print(bv)

jc = eval2(model, compression_loader, entropy_threshold=1.5)
print(jc)

{'accuracy': 11.95, 'avg_entropy': 1.2132835896911136, 'avg_confidence': 0.522}
{'accuracy': 19.83, 'avg_entropy': 0.8805409810345644, 'avg_confidence': 0.685}
{'accuracy': 85.26, 'avg_entropy': 0.22725884611629377, 'avg_confidence': 0.91}
{'accuracy': 57.55, 'avg_entropy': 0.48101341135942255, 'avg_confidence': 0.82}

```

m

```

import matplotlib.pyplot as plt
import torchvision.transforms.functional as F

raw_dataset = datasets.CIFAR10(
    root="data",
    train=False,
    download=True,
    transform=None
)

img_pil, label = raw_dataset[1]

blurred_img = blur(img_pil, radius=2)

def prepare(img):
    return test_transform(img).unsqueeze(0).to(device)

x_clean = prepare(img_pil)
x_blur = prepare(blurred_img)

```



```

model.eval()
with torch.no_grad():
    logits_clean = model(x_clean)
    logits_blur = model(x_blur)

    probs_clean = torch.softmax(logits_clean / 2.0, dim=1)
    probs_blur = torch.softmax(logits_blur / 2.0, dim=1)

    conf_clean, pred_clean = probs_clean.max(dim=1)
    conf_blur, pred_blur = probs_blur.max(dim=1)

fig, axs = plt.subplots(1, 2, figsize=(8, 4))

axs[0].imshow(img_pil)
axs[0].set_title(
    f"Clean\nPred: {pred_clean.item()} | Conf: {conf_clean.item():.2f}"
)
axs[0].axis("off")

axs[1].imshow(blurred_img)
axs[1].set_title(
    f"Blurred\nPred: {pred_blur.item()} | Conf: {conf_blur.item():.2f}"
)
axs[1].axis("off")

plt.show()

```



✎ Visualization

```

from PIL import Image
import matplotlib.pyplot as plt

```

```

CIFAR10_CLASSES = [
    "airplane",
    "automobile",
    "bird",
    "cat",
    "deer",
    "dog",
    "frog",
    "horse",
    "ship",
    "truck"
]

uploaded_img = Image.open("/content/pexels-summerstock-333083.jpg").convert("RGB")
plt.imshow(uploaded_img)
plt.axis("off")

def prepare_image(img):
    return test_transform(img).unsqueeze(0).to(device)

def infer_single_image(model, img, T=2.0, entropy_threshold=1.2):
    model.eval()

    x = prepare_image(img)

    with torch.no_grad():
        logits = model(x)
        probs = torch.softmax(logits / T, dim=1)

        confidence, pred = probs.max(dim=1)
        entropy = -torch.sum(probs * torch.log(probs + 1e-8), dim=1)

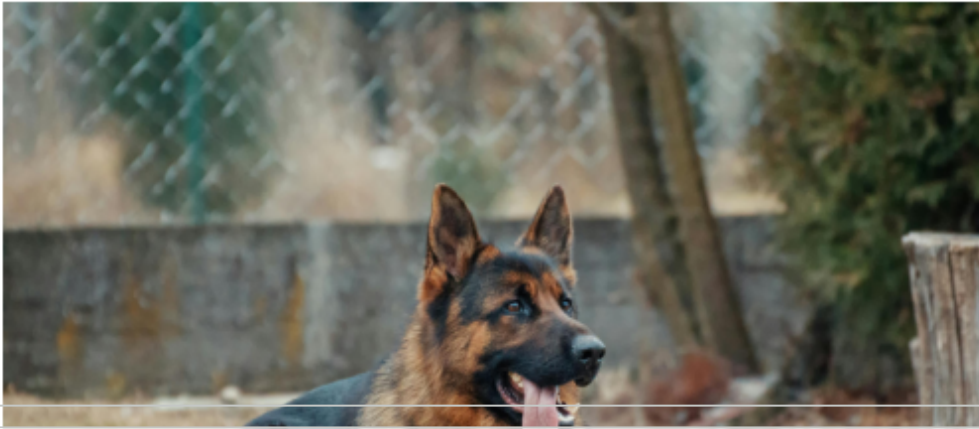
    decision = "accepted"
    if entropy.item() > entropy_threshold:
        decision = "rejected"

    return {
        "prediction": CIFAR10_CLASSES[pred.item()],
        "confidence": confidence.item(),
        "entropy": entropy.item(),
        "decision": decision
    }

result = infer_single_image(model, uploaded_img)
print(result)

```

```
{'prediction': 'dog', 'confidence': 0.8418527245521545, 'entropy': 0.487622976}
```



```
from google.colab import files  
files.download("weights.pth")
```

